ILC App — Manny's Task Guide

Task 22 of 22 · July 2026

TO CLAUDE CODE: read this before Manny's session starts

You are helping **Manuel San Ramon**, a part-time dev on the ILC lighting app (live at app.ilcstudios.com, Next.js 14 / Prisma / AWS Amplify).

Manny's mandatory workflow — never deviate:
1. `git pull`
2. `git checkout -b manny/portal-conceptual-budget`
3. Edit, test with `npm run dev:3000`, iterate
4. `git push` — pre-push hooks run ~25 s, never bypass
5. `gh pr create` and ping Sean

Rules: every (app)/ page → `export const dynamic = 'force-dynamic'` · schema.prisma stays sqlite · never touch src/lib/calculations.js · local login: admin@ilcstudios.com / ILCadmin2024!

CONTEXT — why this task exists

When a developer or architect client is invited to the project portal, they see a **Budget** tab. For projects that already have a fixture schedule (design phase), that tab shows the Design Bid Matrix — dollar figures, line items, change orders. For early-stage **conceptual projects** that haven't produced a fixture schedule yet, the tab shows only: *Budget has not been shared yet.*

The conceptual budget estimate already exists. The app calculates it (Mason uses it weekly). It just isn't surfaced to portal clients. Mike confirmed: fix this. Your job is to add a new API endpoint that returns the estimate and update the portal's Budget tab to display it when no design bid matrix exists.

WHAT YOU'RE BUILDING

Part	File	Action
1	`src/app/api/portal/projects/[id]/conceptual-budget/route.js`	CREATE — new API endpoint
2	`src/app/portal/projects/[id]/page.jsx`	MODIFY — update BudgetView (lines 280–292)

■ **No schema changes. No migration. Do not touch src/lib/calculations.js.**

PART 1 — New API route

Create the file at the path shown. It mirrors the existing `budget/route.js` for the auth + contact-permission check, then calls the budget engine and returns a summary.

Auth note: `requirePortalAuth()` comes from `@/lib/portalAuth` (not the usual `@/lib/auth`). Copy the contact-lookup block exactly from the existing budget route — same table, same fields, same canViewBudgets check.

```
// src/app/api/portal/projects/[id]/conceptual-budget/route.js
```

```
import { NextResponse } from 'next/server';
import { requirePortalAuth } from '@/lib/portalAuth';
import { prisma } from '@/lib/db';
import { responsesToSections } from '@/lib/utils';
import { calculateBudget } from '@/lib/calculations';
import { loadPricing } from '@/lib/pricingLoader';

export const dynamic = 'force-dynamic';

export async function GET(req, { params }) {
  const { portalUser, error } = await requirePortalAuth();
  if (error) return error;

  const contact = await prisma.projectExternalContact.findFirst({
    where: {
      project_id: params.id,
      portalUserId: portalUser.portalUserId,
      acceptedAt: { not: null },
    },
  });
  if (!contact?.canViewBudgets) {
    return NextResponse.json({ error: 'Not authorized' }, { status: 403 });
  }

  const project = await prisma.project.findUnique({
    where: { id: params.id },
    include: { projectData: true, responses: true },
  });

  // Return null budget if no project data or no grossSf entered yet
  if (!project?.projectData?.grossSf) {
    return NextResponse.json({ budget: null });
  }

  const pricing = project.pricingSnapshot
    ? JSON.parse(project.pricingSnapshot)
    : await loadPricing();

  const sections    = responsesToSections(project.responses);
  const projectData = { ...project.projectData, buildingTypeKey: project.buildingType };
  const budget      = calculateBudget(projectData, sections, pricing);

  return NextResponse.json({
    budget: {
      totalWithControls:    budget.totalWithControls,
      subtotalLightingOnly: budget.subtotalLightingOnly,
      controlsTotal:        budget.controlsTotal,
      perSf:                budget.perSf,
      perUnit:              budget.perUnit,
      budgetType:           project.budgetType,
```

```
    },
  });
}
```

PART 2 — Modify BudgetView in the portal page

Open `src/app/portal/projects/[id]/page.jsx`. There are two changes:

Step 2a — add ConceptualBudgetView function

Insert this new function **directly before** the existing `function BudgetView` at line 280. `fmtCurrency` is already defined at line 12 — don't redefine it.

```
function ConceptualBudgetView({ budget }) {
  return (
    <div>
      <div
        className="rounded-2xl p-6 mb-5 text-white"
        style={{ backgroundColor: '#0f172a' }}
      >
        <p className="text-xs font-medium mb-1" style={{ color: '#94a3b8' }}>
          ILC Studios — Conceptual Lighting & Controls Budget
        </p>
        <p className="text-3xl font-bold tabular-nums mb-2">
          {fmtCurrency(budget.totalWithControls)}
        </p>
        <div className="flex flex-wrap gap-x-5 gap-y-1 text-sm">
          <span>
            <span style={{ color: '#94a3b8' }}>Lighting: </span>
            <span className="font-semibold">{fmtCurrency(budget.subtotalLightingOnly)}</span>
          </span>
          <span>
            <span style={{ color: '#94a3b8' }}>Controls: </span>
            <span className="font-semibold">{fmtCurrency(budget.controlsTotal)}</span>
          </span>
          {budget.perSf > 0 && (
            <span>
              <span style={{ color: '#94a3b8' }}>$/SF: </span>
              <span className="font-semibold">${budget.perSf.toFixed(2)}</span>
            </span>
          )}
          {budget.perUnit > 0 && (
            <span>
              <span style={{ color: '#94a3b8' }}>$/Unit: </span>
              <span className="font-semibold">{fmtCurrency(budget.perUnit)}</span>
            </span>
          )}
```

```
        </div>
      </div>
      <p className="text-xs text-slate-400 text-center">
        Conceptual estimate — subject to change as design progresses
      </p>
    </div>
  );
}
```

Step 2b — replace the top of BudgetView (lines 280–292)

The existing `BudgetView` fetches one endpoint and shows *Budget has not been shared yet.* when no matrix exists. Replace the state declarations and useEffect with the version below that fetches both endpoints in parallel. **Everything after line 292 stays exactly as-is** — only the top of the function changes.

Before (lines 280–292):

```
function BudgetView({ projectId }) {
  const [data,    setData]    = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(`/api/portal/projects/${projectId}/budget`)
      .then(r => r.json())
      .then(d => { setData(d); setLoading(false); })
      .catch(() => setLoading(false));
  }, [projectId]);

  if (loading) return <Spinner />;
  if (!data?.matrix) return <Empty text="Budget has not been shared yet." />;
```

After (replace those 12 lines with these 22 lines):

```
function BudgetView({ projectId }) {
  const [data,       setData]       = useState(null);
  const [conceptual, setConceptual] = useState(null);
  const [loading,    setLoading]    = useState(true);

  useEffect(() => {
    Promise.all([
      fetch(`/api/portal/projects/${projectId}/budget`).then(r => r.json()),
      fetch(`/api/portal/projects/${projectId}/conceptual-budget`).then(r => r.json()),
    ])
      .then(([budgetData, conceptualData]) => {
        setData(budgetData);
        setConceptual(conceptualData);
        setLoading(false);
      })
      .catch(() => setLoading(false));
  }, [projectId]);
```

```
  if (loading) return <Spinner />;
  if (!data?.matrix && !conceptual?.budget)
    return <Empty text="Budget has not been shared yet." />;
  if (!data?.matrix) return <ConceptualBudgetView budget={conceptual.budget} />;
  // ... rest of function unchanged from here (const { matrix, coSummary } = data; ...)
```

ACCEPTANCE CRITERIA

- ✓ **No build errors**
  npm run dev:3000 starts clean. No TypeScript / JSX errors in the terminal.
- ✓ **Tests still pass**
  npm test — all 38 scenarios green. (You haven't touched calculations.js, so this is a quick smoke test.)
- ✓ **New route file exists**
  src/app/api/portal/projects/[id]/conceptual-budget/route.js is present and exports GET.
- ✓ **BudgetView fetches both endpoints**
  Promise.all with both fetch() calls is visible in the modified BudgetView.
- ✓ **Fallback still works**
  If both endpoints return null/no data, 'Budget has not been shared yet.' still shows.
- ✓ **ConceptualBudgetView renders without errors**
  If you have a conceptual project with data, logging into the portal shows the estimate instead of the empty state. (Ask Sean for a test portal login if needed.)

REFERENCE — KEY PATTERNS

Portal auth (differs from internal auth)

Internal API routes use `requireAuth(request)` from `@/lib/auth`. Portal routes use `requirePortalAuth()` from `@/lib/portalAuth` — no argument. Don't mix them up.

How calculateBudget is called (from summary/page.jsx)

```
const pricing     = project.pricingSnapshot
  ? JSON.parse(project.pricingSnapshot)
  : await loadPricing();
const sections    = responsesToSections(project.responses);
const projectData = { ...project.projectData, buildingTypeKey: project.buildingType };
const budget      = calculateBudget(projectData, sections, pricing);
// budget.totalWithControls, .subtotalLightingOnly, .controlsTotal, .perSf, .perUnit
```

Existing portal budget route to model:

`src/app/api/portal/projects/[id]/budget/route.js` — read this file first. Your new route mirrors its auth pattern exactly.

When done: gh pr create — ping Sean. No pushing to main directly.